

1. Что такое Docker и для чего он используется?	2
2. Каковы основные преимущества использования Docker?	2
3. Чем контейнер Docker отличается от виртуальной машины?	2
1. Какой командой можно создать и запустить контейнер из образа в Docker?	4
2. Каковы основные компоненты архитектуры Docker?	4
3. Каким образом Docker обеспечивает изоляцию ресурсов для контейнеров?	4
1. Какие возможности предоставляет Dockerfile и как он используется для создания образов контейнеров?	7
2. Как можно реализовать коммуникацию между контейнерами, работающими на одном хосте в Docker?	7
3. Какие методы можно использовать для управления хранением данных в контейнерах Docker?	7
1. Что такое Kubernetes и для чего он используется?	11
2. Какие основные компоненты архитектуры Kubernetes вы знаете?	11
3. Перечислите основные преимущества использования Kubernetes.	11
4. Какие объекты kubernetes вы знаете и для чего они нужны?(прим. pod, service, deployment, etc).....	11
1. Как работает механизм автоматического масштабирования в Kubernetes?	13
2. Какие типы сервисов поддерживает Kubernetes и в чем их различия?	13
3. Как Kubernetes обеспечивает высокую доступность и устойчивость приложений?	13
1. Опишите процесс развертывания приложения с использованием Helm в Kubernetes.....	16
2. Как настроить горизонтальное автоматическое масштабирование подов на основе пользовательских метрик в Kubernetes?	16
3. В чем заключается принцип работы операторов в Kubernetes и как они могут быть использованы для управления состоянием приложения?	16

1. Что такое Docker и для чего он используется?
 2. Каковы основные преимущества использования Docker?
 3. Чем контейнер Docker отличается от виртуальной машины?
-

1. Что такое Docker и для чего он используется?

Docker — это платформа для создания, доставки и запуска приложений в **контейнерах**.

- Контейнер — это **изолированное окружение**, в котором приложение и все его зависимости запускаются одинаково на любой машине.
- Docker упрощает **развёртывание**, тестирование и масштабирование приложений.

Использование:

- Упаковка приложений и зависимостей в контейнер
 - Обеспечение одинакового окружения на dev / test / prod
 - Быстрая доставка и масштабирование микросервисов
-

Пример для интервью

“Docker позволяет разработчику упаковать сервис с его библиотеками и настройками в контейнер, который гарантированно запустится на сервере или в облаке без конфликтов версий.”

2. Основные преимущества использования Docker

1. **Портативность**
 - Контейнеры запускаются одинаково на Windows, Linux, MacOS, облаке.
 2. **Изоляция**
 - Каждый контейнер имеет своё файловое пространство, сеть и процессы.
 3. **Лёгковесность**
 - Делится ядром ОС → контейнеры быстрее и занимают меньше ресурсов, чем VM.
 4. **Быстрая сборка и деплой**
 - Использование Dockerfile + слоёв (layer caching).
 5. **Масштабируемость**
 - Легко запускать несколько экземпляров одного сервиса.
 6. **Управление зависимостями**
 - Всё приложение со всеми библиотеками в одном контейнере.
 7. **DevOps-интеграция**
 - Отлично подходит для CI/CD и оркестраторов (Kubernetes, Docker Swarm).
-

3. Чем контейнер Docker отличается от виртуальной машины?

Характеристика	Docker контейнер	Виртуальная машина (VM)
Изоляция	Процессы изолированы, используют общее ядро ОС	Полная ОС + ядро, изолированная
Вес	Лёгкие, ~МВ–сотни МВ	Тяжёлые, ~GB
Время старта	Секунды	Минуты
Ресурсы	Использует меньше CPU и RAM	Каждая VM требует выделенных ресурсов
Обновление / перенос	Переносимый образ контейнера	Требуется перенос всей VM
Сценарии	Микросервисы, Dev/Test, CI/CD	Полная имитация серверов, legacy-приложения

Итог:

Docker → лёгкая виртуализация на уровне процессов.

VM → полная виртуализация аппаратного окружения.

1. Какой командой можно создать и запустить контейнер из образа в Docker?
 2. Каковы основные компоненты архитектуры Docker?
 3. Каким образом Docker обеспечивает изоляцию ресурсов для контейнеров?
-

1. Создание и запуск контейнера из образа

Основная команда:

```
docker run [опции] <image> [команда]
```

Примеры

▣ Запуск контейнера с интерактивным терминалом:

```
docker run -it ubuntu /bin/bash
```

- `-i` → интерактивный режим
 - `-t` → псевдотерминал
 - `ubuntu` → имя образа
 - `/bin/bash` → команда внутри контейнера
-

▤ Запуск в фоновом режиме (detached):

```
docker run -d -p 8080:80 nginx
```

- `-d` → запуск в фоне
 - `-p 8080:80` → проброс порта хоста на контейнер
 - `nginx` → образ
-

▥ Присвоение имени контейнеру:

```
docker run --name my-container -d nginx
```

2. Основные компоненты архитектуры Docker

1 Docker Engine

- Сердце Docker
 - Состоит из:
 - **Docker Daemon (dockerd)** — сервис, который создаёт и управляет контейнерами
 - **REST API** — позволяет взаимодействовать с daemon программно
 - **CLI (docker)** — командная строка для управления
-

2 Образы (Images)

- Read-only шаблоны для контейнеров
 - Слои (layers) позволяют повторное использование
 - Пример: `ubuntu`, `nginx:latest`
-

3 Контейнеры (Containers)

- Изолированные процессы на базе образа
 - Writable layer сверху read-only слоёв образа
 - Можно запускать, останавливать, удалять
-

4 Registries

- Хранилище образов
 - Public: Docker Hub
 - Private: Harbor, AWS ECR
-

5 Dockerfile

- Скрипт для сборки образа
 - Описывает команды для создания образа
-

6 Volumes / Networks

- **Volumes** → персистентные данные
 - **Networks** → связь между контейнерами и внешним миром
-

3. Как Docker обеспечивает изоляцию ресурсов

Docker использует возможности ядра Linux:

1 Namespaces

- Изолируют:
 - **PID** → процессы контейнера отделены
 - **Network** → свой IP, интерфейсы
 - **Mount** → отдельная файловая система
 - **UTS** → hostname контейнера
 - **IPC** → межпроцессное взаимодействие
 - **User** → свои пользователи и группы
-

2 Control Groups (cgroups)

- Ограничивают ресурсы:
 - CPU
 - Memory
 - I/O
 - Network bandwidth

Пример ограничения памяти при запуске:

```
docker run -m 512m nginx
```

3 Capabilities / Security

- Ограничение системных вызовов
 - Docker по умолчанию запускает контейнер с минимальными правами
 - SELinux / AppArmor обеспечивают дополнительную защиту
-

Короткий ответ для интервью

Docker изолирует контейнеры с помощью namespaces (процессы, сеть, файловая система) и cgroups (ограничение ресурсов), образует контейнер на основе образа и управляется через Docker Engine (daemon + CLI). Контейнер создаётся и запускается командой `docker run`.

1. **Какие возможности предоставляет Dockerfile и как он используется для создания образов контейнеров?**
2. **Как можно реализовать коммуникацию между контейнерами, работающими на одном хосте в Docker?**
3. **Какие методы можно использовать для управления хранением данных в контейнерах Docker?**

1. Возможности Dockerfile и создание образов

Dockerfile — это текстовый файл со списком инструкций, по которым Docker собирает образ контейнера.

Основные возможности:

1. **Указать базовый образ**
2. `FROM ubuntu:22.04`
3. **Выполнить команды**
 - `RUN` → выполняет команды при сборке
4. `RUN apt-get update && apt-get install -y curl`
5. **Копировать файлы в образ**
 - `COPY` → копирует из локальной директории
6. `COPY ./app /app`
 - `ADD` → аналогично `COPY` + умеет разархивировать
7. **Определить рабочую директорию**
8. `WORKDIR /app`
9. **Определить переменные окружения**
10. `ENV PORT=8080`
11. **Открыть порты для контейнера**
12. `EXPOSE 8080`
13. **Определить команду запуска**
14. `CMD ["java", "-jar", "app.jar"]`

или

```
ENTRYPOINT ["python", "app.py"]
```

Пример полного Dockerfile

```
FROM openjdk:17-jdk
WORKDIR /app
COPY target/myapp.jar /app/myapp.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "myapp.jar"]
```

Создание образа

```
docker build -t myapp:1.0 .
```

Запуск контейнера

```
docker run -d -p 8080:8080 myapp:1.0
```

2. Коммуникация между контейнерами на одном хосте

1 Через Docker Bridge Network (по умолчанию)

- Все контейнеры на одном bridge могут обращаться друг к другу через IP

```
docker network create my-network
docker run -d --name db --network my-network postgres
docker run -d --name app --network my-network myapp
```

- В приложении app можно подключаться к db:5432

2 Через пользовательские сети

- Создание сети позволяет использовать **DNS имена контейнеров**

```
docker network create backend-net
docker run -d --name service1 --network backend-net service1-image
docker run -d --name service2 --network backend-net service2-image
```

- service2 может обратиться к service1 по имени service1

3 Через Docker Compose

- Compose автоматически создаёт сеть для сервисов
- Пример docker-compose.yml:

```
services:
  app:
    image: myapp
    ports:
      - "8080:8080"
    depends_on:
      - db
  db:
    image: postgres
```

- app видит db по имени db

3. Управление хранением данных в контейнерах

Контейнеры **stateless**, но часто нужен persistent storage.

1 Volumes (рекомендуется)

- Docker управляет volume, данные сохраняются независимо от контейнера

```
docker volume create mydata
docker run -v mydata:/var/lib/postgresql/data postgres
```

2 Bind Mounts

- Монтирование локальной папки хоста в контейнер

```
docker run -v /home/user/data:/app/data myapp
```

3 tmpfs (in-memory)

- Для временных данных

```
docker run --tmpfs /app/tmp myapp
```

4 Управление через Docker Compose

```
volumes:
  mydata:
services:
  db:
    image: postgres
    volumes:
      - mydata:/var/lib/postgresql/data
```

✓ Плюсы Volumes и Bind Mounts:

- Данные сохраняются после удаления контейнера
- Легко бэкапить / переносить

Итог (ключевые моменты для интервью)

Вопрос	Краткий ответ
Dockerfile	Определяет инструкции для сборки образа: FROM, RUN, COPY, WORKDIR, ENV, EXPOSE, CMD/ENTRYPOINT
Коммуникация контейнеров	Через bridge / пользовательские сети / DNS имена / Docker Compose

Вопрос	Краткий ответ
Хранение данных	Volumes, Bind Mounts, tmpfs; обеспечивает персистентность и независимость контейнера

1. Что такое Kubernetes и для чего он используется?
 2. Какие основные компоненты архитектуры Kubernetes вы знаете?
 3. Перечислите основные преимущества использования Kubernetes.
 4. Какие объекты kubernetes вы знаете и для чего они нужны?(прим. pod, service, deployment, etc)
-

1. Что такое Kubernetes и для чего он используется

Kubernetes (k8s) — это платформа для оркестрации контейнеров, которая автоматизирует:

- развертывание контейнеров,
- масштабирование,
- управление ресурсами,
- самовосстановление (self-healing),
- сетевое взаимодействие между сервисами.

Использование:

- Управление кластерами контейнеров в продакшене.
 - Масштабирование микросервисов под нагрузкой.
 - Автоматизация деплоя и обновлений.
 - Оркестрация сетевого взаимодействия между контейнерами.
-

Короткий ответ для интервью

Kubernetes — это система управления контейнеризованными приложениями, обеспечивающая автоматическое развертывание, масштабирование, балансировку нагрузки и отказоустойчивость.

2. Основные компоненты архитектуры Kubernetes

◆ Control Plane (плоскость управления)

- **kube-apiserver** — API, через который управляется кластер.
- **etcd** — распределённое key-value хранилище состояния кластера.
- **kube-scheduler** — распределяет Pods по узлам.
- **kube-controller-manager** — контроллеры для поддержания желаемого состояния (replication, node, endpoints).
- **cloud-controller-manager** — интеграция с облачной инфраструктурой.

◆ Node (рабочие узлы)

- **kubelet** — агент, который следит за Pod'ами на узле.
 - **kube-proxy** — обеспечивает сетевое взаимодействие и балансировку.
 - **Container Runtime** — Docker, containerd, CRI-O.
-

◆ Объекты API (для работы)

- Pod, Service, Deployment, ConfigMap, Secret, Ingress, StatefulSet, DaemonSet, Job, CronJob.

3. Основные преимущества использования Kubernetes

1. **Масштабируемость** — вертикальная и горизонтальная.
2. **Self-healing** — автоматический перезапуск упавших контейнеров.
3. **Балансировка нагрузки** — через Service и Ingress.
4. **Декларативное управление** — описываем desired state, Kubernetes поддерживает его.
5. **Автоматические обновления и откаты** — через Deployments.
6. **Универсальность и портативность** — работает в облаке, on-prem, hybrid.
7. **Секреты и конфигурация** — безопасное хранение данных и конфигов.

4. Основные объекты Kubernetes и их назначение

Объект	Назначение
Pod	Минимальная единица деплоя, может содержать один или несколько контейнеров, общий IP и тома
Deployment	Управляет Pod'ами, поддерживает обновления, откаты, масштабирование
Service	Абстракция для доступа к Pod'ам (ClusterIP, NodePort, LoadBalancer)
ConfigMap	Хранение конфигурационных данных в виде key-value
Secret	Хранение чувствительных данных (пароли, токены)
StatefulSet	Для приложений с состоянием (базы данных), обеспечивает стабильные идентификаторы Pod'ов
DaemonSet	Обеспечивает запуск Pod'ов на всех узлах кластера
Job / CronJob	Запуск разовых / периодических задач
Ingress	Управление внешним доступом (HTTP/HTTPS) к сервисам
Volume / PersistentVolume / PersistentVolumeClaim	Организация персистентного хранения данных

1. Как работает механизм автоматического масштабирования в Kubernetes?
 2. Какие типы сервисов поддерживает Kubernetes и в чем их различия?
 3. Как Kubernetes обеспечивает высокую доступность и устойчивость приложений?
-

1. Механизм автоматического масштабирования в Kubernetes

Kubernetes поддерживает 3 типа масштабирования:

1 Horizontal Pod Autoscaler (HPA)

- **Что делает:** увеличивает или уменьшает количество Pod'ов в Deployment/ReplicaSet в зависимости от нагрузки.
- **Метрики:** CPU, память, пользовательские метрики (custom metrics), запросы через Prometheus.
- **Пример:**

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: myapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
```

HPA следит за метриками и динамически регулирует количество Pod'ов.

2 Vertical Pod Autoscaler (VPA)

- **Что делает:** автоматически увеличивает/уменьшает ресурсы (CPU, память) для Pod'ов.
 - **Используется:** для stateful приложений, где масштабирование горизонтально не всегда возможно.
-

3 Cluster Autoscaler

- **Что делает:** добавляет или удаляет узлы (nodes) в кластере, если не хватает ресурсов для Pod'ов.
 - **Используется:** облачные среды (AWS, GCP, Azure).
-

2. Типы сервисов Kubernetes и их различия

Тип сервиса	Назначение	Особенности
ClusterIP (по умолчанию)	Внутренний доступ к Pod'ам внутри кластера	Не доступен извне
NodePort	Доступ к Pod'ам через порт на всех узлах	Простое решение, внешние клиенты → <NodeIP>:<NodePort>
LoadBalancer	Интеграция с облачным LB	Создаёт внешнюю точку доступа с балансировкой
ExternalName	Прокси к внешнему DNS-ресурсу	Не создаёт Pod или Endpoint, просто DNS alias
Headless Service (ClusterIP: None)	Для stateful приложений	Дает Pod'ам фиксированные DNS имена, полезно для StatefulSet

3. Как Kubernetes обеспечивает высокую доступность и устойчивость

1 Self-healing

- **kubelet** и **Deployment/ReplicaSet** автоматически перезапускают упавшие Pod'ы.
- Pod'ы на недоступных узлах пересоздаются на других узлах.

2 ReplicaSets / Deployments

- Обеспечивают **минимальное количество Pod'ов**, автоматическое восстановление и rolling update.

3 Health Checks

- **Liveness Probe:** перезапуск Pod'а при сбое приложения.
- **Readiness Probe:** контролирует доступность Pod'а для сервиса.

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

4 Anti-affinity / Node selection

- Распределение Pod'ов по разным узлам для отказоустойчивости.
- Пример:

```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: app
              operator: In
              values:
                - myapp
        topologyKey: "kubernetes.io/hostname"
```

5 Persistent Volumes + StatefulSet

- Хранение данных вне Pod'ов → Pod можно пересоздать без потери данных.

Кратко для интервью

Kubernetes обеспечивает высокую доступность через ReplicaSets, self-healing, health checks, распределение по узлам, и масштабирование с помощью HPA, VPA и Cluster Autoscaler. Сервисы позволяют управлять доступом внутри кластера и извне через ClusterIP, NodePort и LoadBalancer.

1. Опишите процесс развертывания приложения с использованием Helm в Kubernetes.
 2. Как настроить горизонтальное автоматическое масштабирование подов на основе пользовательских метрик в Kubernetes?
 3. В чем заключается принцип работы операторов в Kubernetes и как они могут быть использованы для управления состоянием приложения?
-

1. Развертывание приложения с использованием Helm

Helm — это пакетный менеджер для Kubernetes, позволяющий управлять приложениями через **чарт (chart)** — набор YAML-шаблонов + значения (`values.yaml`).

Процесс развертывания:

1 Установка Helm

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |  
bash
```

2 Создание чарта

```
helm create myapp
```

- Генерируется структура: `templates/`, `values.yaml`, `Chart.yaml`

3 Настройка значений

- Определяем параметры в `values.yaml` (образ, порты, реплики, ресурсы)

```
replicaCount: 2  
image:  
  repository: myapp  
  tag: "1.0.0"  
service:  
  type: ClusterIP  
  port: 8080  
resources:  
  limits:  
    cpu: "500m"  
    memory: "512Mi"
```

4 Установка чарта в кластер

```
helm install myapp ./myapp
```

5 Обновление приложения

```
helm upgrade myapp ./myapp
```

6 Удаление

```
helm uninstall myapp
```

✓ Преимущества:

- Декларативное управление
- Параметризация через `values.yaml`
- Поддержка версииности и `rollback` (`helm rollback`)

2. Горизонтальное масштабирование подов на основе пользовательских метрик

Horizontal Pod Autoscaler (HPA) с custom metrics

1 Установка Metrics Server

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

2 Проверка доступных метрик

```
kubectl top pod
```

3 HPA с пользовательской метрикой

- Требуется Prometheus Adapter для custom metrics
- Пример HPA:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: myapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Pods
    pods:
      metric:
        name: requests_per_second
      target:
        type: AverageValue
        averageValue: 100
```

- Kubernetes автоматически масштабирует Pod'ы, если метрика `requests_per_second` превышает 100.

3. Принцип работы операторов (Operators) в Kubernetes

Что такое оператор

- **Operator** — это контроллер, который **расширяет Kubernetes API** для управления приложениями с состоянием (stateful applications).
 - Инкапсулирует **знание о жизненном цикле приложения**: установка, обновление, бэкапы, восстановление.
-

Как работают операторы

1. Custom Resource Definition (CRD)

- Создаёт новый тип ресурса, например MySQLCluster

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: mysqlclusters.mycompany.com
spec:
  group: mycompany.com
  versions:
    - name: v1
      served: true
      storage: true
  scope: Namespaced
  names:
    plural: mysqlclusters
    singular: mysqlcluster
    kind: MySQLCluster
```

2. Custom Controller

- Следит за CRD и поддерживает desired state
- Автоматически выполняет действия:
 - Создаёт StatefulSet и Service
 - Делает бэкап базы
 - Восстанавливает Pod'ы при сбое

3. Пример использования

- Operator MySQL создаёт и управляет кластером MySQL полностью через CRD:

```
apiVersion: mycompany.com/v1
kind: MySQLCluster
metadata:
  name: mysql-prod
spec:
  replicas: 3
  version: 8.0
```

Преимущества операторов

- Автоматизация управления сложными stateful-приложениями
 - Поддержка обновлений и откатов
 - Умный self-healing на уровне приложения, а не только Pod
-

Коротко для интервью

Тема	Ключевой момент
Helm	Пакетный менеджер для Kubernetes, управление приложениями через чарт + <code>values.yaml</code>
HPA на пользовательских метриках	Масштабирование Pod'ов с помощью HPA и custom metrics через Prometheus Adapter
Operators	Контроллер + CRD, автоматизация жизненного цикла сложных приложений (stateful)